Same Label, Different Work: Roles, Resolution Time and Topics of GitHub Issues Attributed to Thirteen AI Coding Agents

Bernard Barnieh
Trent University
Peterborough, Ontario, Canada

Abstract. AI coding agents no longer only write code: they open, trigger and appear in GitHub issues, yet empirical studies of these agents have focused on their pull requests. This paper presents, to the researcher's knowledge, the first comparison of the issues attributed to many agents. Following the Goal–Question–Metric approach, it analyses 4,952 issues from 3,466 repositories attributed to 13 agents, combining non-parametric tests with effect sizes, survival analysis that accounts for issues still open, and topic models checked for stability across runs and confirmed with a second method. The attribution, the topic labels and the language flag were also validated by hand on blind random samples. The agents differ most in how an issue is linked to them. For CodeRabbit and Devin, most issues were written by the agent's own bot, and these are closed 18 to 24 percentage points less often than the same agents' other issues, a gap that issue age does not explain. Time-to-close differs across agents with a medium effect, but rankings based on closed issues alone disagree with the survival analysis. Topics also depend on the agent, most clearly for CodeRabbit's review follow-ups. The manual check found 85.9% of attributions correct, with the agent only mentioned in half of the correct ones. The label "AI agent" therefore covers distinct kinds of work, and studies that compare agents should separate the agent's role, measure speed with survival analysis, and validate attributions and topics by hand.

Keywords. AI coding agents, GitHub issues, mining software repositories, survival analysis, topic modelling, empirical software engineering

I. Introduction

AI coding agents have moved from suggesting code to acting in repositories on their own. Systems such as SWE-agent take a GitHub issue as input and try to resolve it end to end [1], and the benchmark most often used to evaluate them, SWE-bench, is itself built from real GitHub issues and the pull requests that closed them [2]. Developers adopt these tools quickly: in a controlled experiment, programmers with GitHub Copilot finished a task 55.8% faster than those without it [3], and AIDev, a recent dataset of agent activity on GitHub, contains hundreds of thousands of pull requests opened by agents [4]. The issue tracker sits at the start of this workflow: it is where work is requested, discussed and planned, and its use in open-source projects has been studied at scale [5].

Agents now also appear *inside* issue trackers, and they do so in different roles. A review bot such as CodeRabbit opens follow-up issues for problems it found in a pull request. Sweep is summoned by a human who adds a label or a title prefix. Other agents show up only because someone mentions or discusses them. These roles matter to maintainers. When projects adopted earlier code-review bots, their pull-request activity changed: more pull requests were merged and developers communicated less [6], and developers describe unwanted bot activity as noise that distracts them from their work [7]. For researchers, the roles create a measurement problem: datasets attribute issues to agents with heuristic signals, and the most recent of these datasets warns that its agent label is a candidate attribution, not a verified one [8]. If "issues attributed to agent X" mixes issues the agent wrote with issues that merely mention it, comparisons between agents may say more about the attribution than about the agents.

This paper compares the GitHub issues attributed to 13 AI coding agents. Following the Goal–Question–Metric approach [9], it asks three research questions:

- **RQ1**: *How do issues attributed to different agents differ in volume, state and labelling?* Before comparing speed or content, maintainers need a baseline of what each agent's issues look like and how often they are closed.
- **RQ2**: *Does the time-to-close of these issues differ across agents, and by how much?* Time-to-close is a common proxy for how quickly maintainers act on an issue [10], but a third of the issues in the data are still open, so the answer depends on how open issues are treated.
- **RQ3**: *Which topics do the issues raise, and does that differ by agent?* Topics show what kind of work an agent's issues represent, which helps explain the differences found in RQ1 and RQ2.

Most empirical work on coding agents studies their pull requests: how often they are accepted [11], [12], what tasks developers hand to them [13], and how their changes and descriptions differ from those of humans [14]. Closer to the setting of this study, Ghaleb showed that agents leave behavioural fingerprints in their pull requests [15], and Selvanayagam and Ghaleb studied agents that review other agents' code [16]. Sayagh asked which issues make good tasks for an agent [17], but for a single agent. To the best of the researcher's knowledge, no study has compared the issues attributed to many agents on their characteristics, their resolution time and their topics together.

Submitted for COIS-5210H Advanced Topics in Software Engineering, Individual Assignment 1, Trent University, October 2026.

The study covers 4,952 issues from 3,466 repositories, attributed to 13 agents. The data turned out to be less uniform than the label suggests. For CodeRabbit and Devin, most issues were written by the agent's own bot account (84.6% and 81.6%), while for Claude Code, Sweep and Jules almost none were. This role is associated with the outcome: within the same agent, bot-written issues are closed less often than the others, by 18 to 24 percentage points. How open issues are handled also matters. Ranking the agents by the time-to-close of their closed issues gives a very different order from a survival analysis that counts the issues still open (Spearman $\rho = 0.22$). The topics confirm the picture from a different angle. CodeRabbit's own issues are follow-ups from pull-request reviews, with code-level fixes as their largest topic, and this result holds under two different topic models, even though the two models divide the corpus differently.

The paper contributes the following:

1) a comparison of issues attributed to 13 AI coding agents on volume, state, labelling, time-to-close and topics;
2) evidence that the role an agent plays in an issue, written by its bot, triggered by a human or merely mentioned, differs sharply between agents and is associated with whether the issue is closed;
3) a demonstration that comparing agents by the closed issues alone can reverse their ranking, and that survival analysis avoids this;
4) a manual, blind validation of the attribution, the topic labels and the language flag on random samples, and a replication package with all scripts, independent verification checks and the validation answers.

Paper organisation. Section II reviews related work. Section III describes the data and the methodology. Sections IV to VI answer the three research questions. Section VII discusses implications, Section VIII the threats to validity, and Section IX concludes.

II. Background and Related Work

AI coding agents on GitHub. Research on coding agents has moved quickly from benchmarks to field data. SWE-bench measures whether a model can resolve real GitHub issues [2], and SWE-agent showed that a purpose-built interface lets a language model navigate a repository, edit files and run tests to do so [1]. Field studies now follow agents into real projects. Li et al. describe the arrival of autonomous agents as "AI teammates" and released AIDev, a large dataset of agent-authored pull requests; agent pull requests are accepted less often than human ones [4], [11]. Pinna et al. compared five agents and found that acceptance depends strongly on the type of task, so that a single global ranking of agents can mislead [12]. Watanabe et al. studied pull requests made with Claude Code and found that developers often delegate refactoring, documentation and testing to it [13]. Pham and Ghaleb compared the code changes and descriptions of agentic and human pull requests [14], and Selvanayagam and Ghaleb studied pull requests in which one agent reviews another agent's code [16]. These studies focus on pull requests. Sayagh is an exception: the study examines which GitHub issues lead to merged pull requests by the Copilot agent [17], but it treats the issue as input for one agent. The present study looks instead at the issues attributed to thirteen agents, whatever role the agent plays in them.

Attributing activity to agents and bots. Knowing who or what produced an artefact is a research problem of its own. Bot accounts are common on GitHub: Wessel et al. found bots in 26% of the popular projects they sampled [18], and Golzadeh et al. built a ground-truth dataset and classifier to separate bot from human accounts in issue and pull-request comments [19]. For coding agents the problem is harder, because agent output is often posted under human accounts. Ghaleb showed that agents leave fingerprints that a classifier can use to identify them [15], and the CodAGE dataset, from which the data of this study appear to derive, assigns agents with login, branch and text signals and calls the result a candidate attribution with false positives by design [8]. Kalliamvakou et al. warned more generally that GitHub data contain many such perils for researchers [20]. The analysis here carries that warning down to single issues: it records how each issue is linked to its agent, and the link turns out to differ sharply between agents.

Bots in maintainers' work. Bots change how projects work. When projects adopted code-review bots, more pull requests were merged and developers communicated less [6]. Practitioners nevertheless disagree about what counts as a development bot and what it is good for [21], and developers describe unwanted bot activity as noise that disrupts their work [7]. Agents that open their own issues add to this activity. Whether maintainers act on those issues has, as far as the researcher is aware, not been measured; RQ1 and RQ2 do so.

Issue labels, lifetime and topics. Labels are GitHub's built-in way to classify and triage issues. Their use varies between projects and is associated with how issues are resolved [22], and tools such as Ticket Tagger assign them automatically [23]. Labels and issue types are noisy, however: Herzig et al. inspected more than 7,000 issue reports by hand and found that about a third were misclassified [24]. The lifetime of an issue has been studied as a prediction target [10]. To learn what issues are about, mining studies most often use LDA [25]–[27], although its topics can change with the random seed [28], [29]. Recent studies of agent pull requests and of ChatGPT use in issue trackers use BERTopic instead [30]–[32]. Labels, lifetime and topics are combined here; open issues are handled with survival analysis, and topics are reported only after their stability has been tested and a second topic model has confirmed them.

III. Data and Methodology

A. Study Design

The study answers RQ Set 5 (agent type) of the assignment, whose three questions concern issue traits (volume, state and labels), time-to-close, and issue topics, each compared across agent types; RQ1 to RQ3 refine these three questions in that order. The researcher designed the study with the Goal–Question–Metric approach [9]. Its goal is to *analyse* GitHub

TABLE I
RESEARCH QUESTIONS AND METRICS (GQM). A = ATOMIC, C = COMPOSITE

ID	Metric	Type
RQ1 How do issues differ in volume, state and labelling?		
M1.1	Issues per agent	A
M1.2	Repositories per agent; issues per repository; Gini	C
M1.3	Share of issues closed, with Wilson 95% CI	C
M1.4	Closure reason: completed, not planned, duplicate	C
M1.5	Share of issues with at least one label	C
M1.6	Labels per issue	A
M1.7	Label categories; five most frequent labels	C
M1.8	χ^2 and Cramér's V: agent × state and agent × labelled	C
M1.9	Language group mix; χ^2 and V	C
M1.10	Attribution signal mix; χ^2 and V	C
RQ2 Does time-to-close (TTC) differ across agents, and by how much?		
M2.1	TTC in hours, from the GitHub API timestamps	A
M2.2	Median with exact 95% CI, IQR, mean, SD	C
M2.3	Normality: Shapiro–Wilk, skewness, Q–Q plots	C
M2.4	Kruskal–Wallis test and ε^2	C
M2.5	Dunn's test (Holm) and Cliff's δ per pair of agents	C
M2.6	Kaplan–Meier median with open issues censored; log-rank test	C
M2.7	TTC by closure reason	C
RQ3 Which topics do the issues raise, and does that differ by agent?		
M3.1	Topic quality: Cv coherence and reproducibility across seeds	C
M3.2	Top words and hand-assigned label per topic	C
M3.3	Topic share per agent, with bootstrap 95% CI	C
M3.4	χ^2 and Cramér's V: agent × dominant topic	C

issues attributed to AI coding agents *for the purpose of* understanding how these issues differ between agents, *with respect to* their characteristics, their resolution time and their topics, *from the viewpoint of* open-source maintainers and software-engineering researchers, *in the context of* 4,952 public issues from 3,466 repositories attributed to 13 agents. Table I refines this goal into the three research questions and 21 metrics; each metric is either atomic (A), read directly from the data, or composite (C), derived from other values.

Statistical analysis. The data are counts and heavily skewed durations, so all tests used are free of normality assumptions, and every test is reported with an effect size. Tables of counts are tested with Pearson's χ^2 test, with the p-value computed by permutation (10,000 permutations) because up to 36% of the expected cell counts are below five, which makes the usual approximation unreliable. Cramér's V [33] measures the strength of an association and is judged with Cohen's benchmarks, adjusted for the degrees of freedom of the table [34]; adjusted standardised residuals show which cells drive an association. Proportions carry Wilson intervals [35]. Time-to-close is compared with the Kruskal–Wallis test [36] and its effect size ε^2 [37], followed by Dunn's test [38] and Cliff's δ [39], interpreted with the thresholds of Romano et al. [40]. Medians carry exact, distribution-free confidence intervals from order statistics [41]. Because many issues are still open, time-to-close is also estimated with the Kaplan–Meier estimator [42], which counts an open issue as censored rather than ignoring it, and agents are compared with the log-rank test [43]. Each family of tests is corrected with Holm's method [44]. This is the same family of tests the closest prior work uses for agent comparisons (χ^2 with Cramér's V and Cliff's δ [16]; Kruskal–Wallis with Dunn's test and ε^2 [14]), extended with normality checks, multiple-comparison correction and censoring. Statistical claims are made only for the eleven agents with at least 30 closed issues.

Sensitivity analyses. Because the agent attribution is heuristic, `agent_type` is kept as given in the main analysis, as the study brief requires, and the key tests are repeated on four subsets: issues created after the agent's public release (S1); human-authored issues only (S2); the high-precision subset recommended by the CodAGE documentation, which drops automation echoes and repositories named after the agent (S3); and one value per agent–repository pair, so that busy repositories cannot dominate (S4).

B. Data Source and Processing

Data source. The course dataset contains 5,000 GitHub issues from 3,477 repositories, each with an `agent_type` label. It appears to be derived from CodAGE [8], a dataset of agent-related GitHub events built from GH Archive, here enriched with live fields from the GitHub API. This is the researcher's inference rather than a statement of the brief: the agent names and the attribution scheme match, and the latest GH Archive event in the data falls on CodAGE's documented end date. The inference matters for interpretation, because CodAGE favours recall over precision and describes its agent label as a candidate attribution with false positives by design.

Cleaning. Fifteen documented decisions were applied (Table II); every step records its row count, and the raw files are never changed. The cleaning removed 18 pull requests that were stored as issues, and 30 re-submitted duplicates, that is, issues with the same repository and the same normalised title and body posted within 24 hours, of which the first is kept. Issues that maintainers *closed as* duplicates stay in the data, since they are real maintainer decisions. Time-to-close was recomputed from the API timestamps, because the archive-based fields were stale, and all durations were kept, including 47 issues closed within a minute and 119 that took more than a year: trimming them would hide exactly the behaviour under study. The 1,677 open issues (33.9%) are kept and treated as censored at the most recent API update in the data. The result is 4,952 issues from 3,466 repositories. Copilot and Cursor have three issues each and therefore appear only in descriptive results.

Attribution signal. Since the agent label is a candidate attribution, the study records *how* each issue is linked to its agent, following the precision recipe in the CodAGE documentation; Selvanayagam and Ghaleb similarly separated high-confidence signatures from weaker ones when attributing pull requests [16]. An issue is either written by the agent's own bot account (2,091 issues, 42.2%), re-posted by `github-actions[bot]` (134), filed in a repository whose name contains the agent's name, such as the agent's own project (189), or linked by another signal, such as a mention or a trigger keyword (the remainder). Bot logins are the most

TABLE II
DATA PROCESSING DECISIONS; THE FULL LOG IS IN THE REPLICATION PACKAGE

D1	Raw files read-only; every step logged with row counts
D2	Pull requests removed (−18)
D3	Re-submitted duplicates removed (−30); issues *closed as* duplicate kept
D4	Time-to-close from API timestamps, in hours
D5	No outlier trimming (47 closed within a minute, 119 after over a year)
D6	Open issues kept and censored (1,677)
D7	Statistical tests only for agents with ≥ 30 closed issues (11 agents)
D8	`agent_type` kept as given; flags for bot authors (2,278) and pre-release issues (10)
D9	Labels lower-cased and mapped to five categories by published rules
D10	Text = title + body; text cleaning only for RQ3
D11	Comment threads not used (no metric needs them)
D12	Censoring date = latest API update (2026-10-09)
D13	Language flag: 695 non-Latin script, 48 Latin-script non-English
D14	Renamed repositories and transferred issues flagged
D15	Attribution signal: own bot 2,091, Actions echo 134, self-referential 189

reliable of these signals, while text signals are the main source of false positives [8].

Language. Topic modelling needs a single language, and Pham and Ghaleb likewise kept only English descriptions in their study of agent pull requests [14]. A script and function-word heuristic flags each issue as non-Latin script, Latin-script non-English or English. All issues stay in RQ1 and RQ2; RQ3 uses the English ones.

Manual validation. Three variables come from heuristics, so the researcher validated them by hand on random samples drawn with a fixed seed: the attribution (99 issues, nine per tested agent: is the agent really involved, and in which role?), the RQ3 topic labels (five issues per topic) and the language flag (40 issues). The samples are blind, that is, they do not show the study's own classification, and the answers are saved in the replication package so that others can re-rate them.

IV. RQ1: How Do Issues Differ in Volume, State and Labelling?

Motivation. Before asking how fast agents' issues are closed or what they are about, a baseline is needed: how many issues each agent has, whether they get closed, how they are labelled, and how they are linked to the agent in the first place. Maintainers who consider adopting an agent face the same questions.

Approach. Metrics M1.1–M1.10 were computed for every agent (Table III), and five associations were tested between the agent and a category (state, closure reason, labelled or not, language group and attribution signal) on the eleven tested agents. Adjusted residuals then show which agents drive each association. For the two agents with both bot-written and other issues, CodeRabbit and Devin, closure rates were compared within the agent with a two-proportion test, Newcombe intervals for the difference [45] and Cohen's h [34].

Results. *Volume and spread.* Three agents dominate the data: CodeRabbit, Claude Code and Sweep account for 70.7% of all issues, while the five smallest tested agents have fewer than 110 issues each. The issues are spread thinly over many repositories. Every tested agent has a median of one issue per repository, and no single repository holds more than 12.7% of any tested agent's issues, so no project dominates an agent's profile; the Gini coefficient of each agent's issues over repositories ranges from 0.04 (Aider) to 0.40 (Devin). Closure rates vary widely, from 36.4% for Amazon Q (95% CI [27.9, 45.9]) to 79.9% for Gemini Code Assist (95% CI [74.6, 84.4]).

TABLE III
AGENT PROFILE (RQ1). CLOSED, LABELLED, NON-EN. (NOT ENGLISH) AND OWN BOT ARE % OF THE AGENT'S ISSUES; NP = CLOSED AS NOT PLANNED, % OF CLOSED ISSUES

Agent	Issues	Repos	Closed	NP	Labelled	Non-En.	Own bot
CodeRabbit	1,942	1,300	59.7	12.2	35.4	12.0	84.6
Claude Code	876	619	79.2	6.8	54.5	20.3	0.9
Sweep	685	572	67.6	9.3	82.0	2.2	0.1
Devin	538	267	57.2	18.5	40.1	33.8	81.6
SWE-agent	309	231	78.6	14.4	49.2	6.8	0.0
Gemini Code Assist	254	201	79.9	13.3	63.4	33.1	0.0
Amazon Q	107	95	36.4	15.4	69.2	7.5	0.0
Aider	68	65	66.2	6.7	47.1	4.4	0.0
Kiro	63	59	73.0	8.7	60.3	9.5	0.0
PR-Agent	55	41	54.5	13.3	29.1	9.1	0.0
Jules	49	43	77.6	5.3	67.3	14.3	0.0
Copilot†	3	3	100.0	0.0	33.3	0.0	0.0
Cursor†	3	3	100.0	0.0	100.0	0.0	0.0
All	4,952	3,466	66.1	11.3	49.5	15.0	42.2

†Descriptive only (three issues each).

Associations. All five associations are significant after Holm correction ($p_{\text{perm}} \leq 0.001$), but their strength differs a great deal. The agent is weakly associated with state (V = 0.21, small), closure reason (V = 0.11, small) and language (V = 0.21, small), moderately with labelling (V = 0.33, medium; on average an issue carries from 0.55 labels for PR-Agent to 2.01 for Amazon Q), and strongly with the attribution signal (V = 0.54, large). The clearest difference between agents, then, is not what happens to their issues but how the issues are tied to them in the first place (Fig. 1). For CodeRabbit (84.6%) and Devin (81.6%), the agent's own bot account wrote most issues. For Claude Code, Sweep and Jules, almost none were bot-written: a human opened the issue and the agent is triggered, mentioned or credited in the text. The manual check of a random sample of 99 issues shows how varied these links are: of the issues that are really linked to their agent, the agent wrote 29.4%, a person triggered the agent in 20.0%, and in 50.6% the agent is only mentioned or discussed. Language differs as well: 33.8% of Devin's issues are not in English, against 2.2% of Sweep's.

Finding 1: agent-written issues stay open. The role of the agent turns out to matter for whether an issue is closed. Within the same agent, issues written by the agent's own bot are closed less often than issues linked to the agent in other ways (Table IV): −18.3 percentage points for CodeRabbit (95% CI [−23.5, −12.6]; h = 0.39, small) and −23.9 points for Devin (95% CI [−32.5, −13.6]; h = 0.51, medium), both significant after Holm correction. Issue age is an obvious alternative explanation, but it does not hold: the bot-written issues are

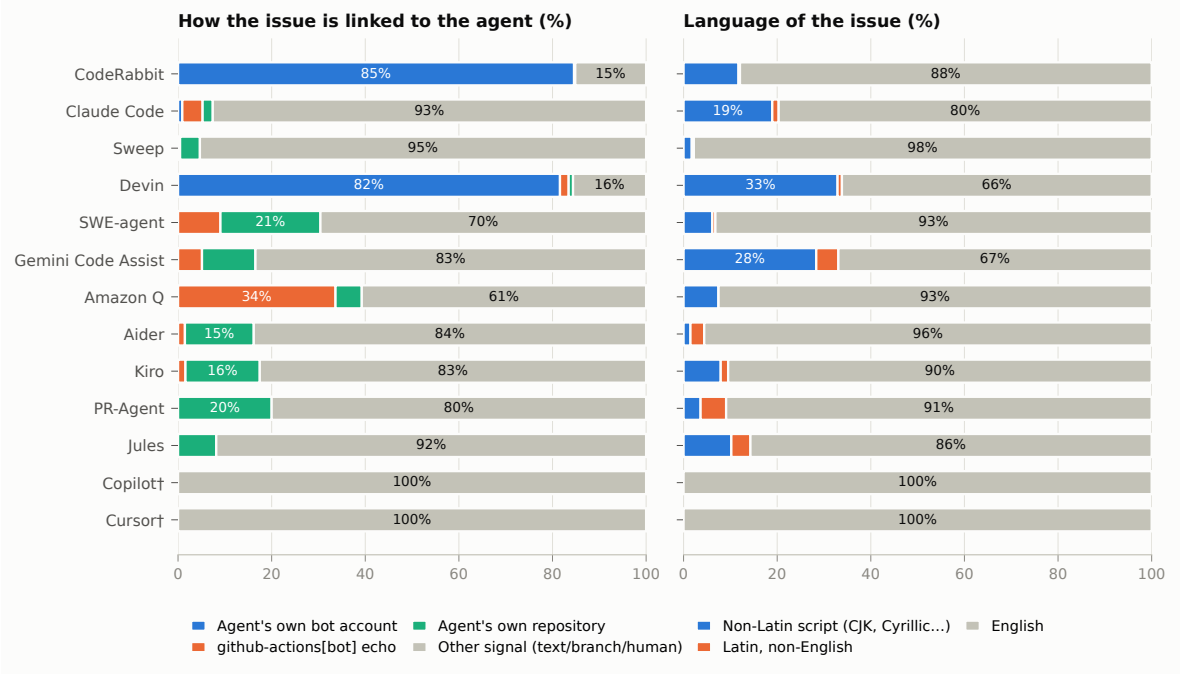


Fig. 1. How each agent's issues are linked to the agent (left) and their language (right).

TABLE IV
CLOSURE RATE (%, WILSON 95% CI) OF ISSUES WRITTEN BY THE AGENT'S OWN BOT AND OF THE AGENT'S OTHER ISSUES

Agent	Own bot	Other	Diff. (pp)	z	p
CodeRabbit	56.9 [54.5, 59.3]	75.3 [70.1, 79.8]	−18.3	−5.95	2.7×10^{-9}
Devin	52.8 [48.2, 57.5]	76.8 [67.5, 84.0]	−23.9	−4.35	1.4×10^{-5}

older (median 403 against 233 days for CodeRabbit), so they have had more time to be closed, and among issues at least 180 days old the gaps hardly change (−18.1 and −24.8 points). Across all agents, 56.0% of bot-written and 73.5% of other issues are closed. The bot-written issues of the two agents are of different kinds. CodeRabbit's are follow-ups from its reviews (92.6% refer to a pull request or review), whereas Devin's mostly describe new tasks (12.8%). For CodeRabbit, a plausible reading is that maintainers treat review follow-ups as lower-priority suggestions, in line with developers' reports that unsolicited bot activity feels like noise [7]. The data show the association, not its cause.

Finding 2: the labelling illusion. Labels are GitHub's built-in way to classify and triage issues, and how projects use them is associated with how issues are resolved [22]. By the usual measure, Sweep is the most-labelled tested agent: 82.0% of its issues carry a label (95% CI [79.0, 84.7]). Most of these labels, however, are `sweep` itself (61.7% of all labels on Sweep's issues), the label a user adds to invoke the agent. Counting only labels that classify the work, Sweep falls to 19.1% and ranks last of all thirteen agents, a drop of 62.9 points. Amazon Q (−9.3 points) and Jules (−10.2) drop too, while no other agent changes by more than 1.5 points. For trigger-based agents, a label therefore records how the agent was called, not how maintainers organise the work. This adds a new source of noise to labels that are already known to be unreliable as an issue classification [24].

RQ1 Summary. Agents differ most in *how* their issues are linked to them (V = 0.54). *Finding 1:* for CodeRabbit and Devin, issues opened by the agent's own bot are closed 18–24 percentage points less often than the same agents' other issues, and issue age does not explain the gap. *Takeaway:* maintainers who enable a review agent should triage or expire its follow-up issues, and researchers should report the agent's role rather than compare agents as a single group. *Finding 2:* for trigger-based agents such as Sweep, most labels invoke the agent rather than classify the work. *Takeaway:* agent-workflow labels must be removed before labels are used as data or for triage dashboards.

V. RQ2: DOES TIME-TO-CLOSE DIFFER ACROSS AGENTS?

Motivation. Time-to-close is a common proxy for how quickly maintainers act on an issue, and it has long been studied as a property of issues [10]. For an adopter, it answers a practical question: do the issues that come with one agent sit idle longer than those of another? It also comes with a pitfall. A third of the issues in the data are still open, and any analysis that uses only closed issues leaves out the slowest ones.

Approach. The first comparison uses the 3,269 closed issues of the eleven tested agents. Their time-to-close is far from normal: Shapiro–Wilk [46] rejects normality for every agent ($p < 10^{-6}$; skewness 2.0 to 13.0), and even after a log transformation it is rejected for 9 of the eleven agents. The right tail is long: across the tested agents, the mean time-to-close is 1,178.3 hours against a median of 50.6 hours (IQR 686.2 h, SD 3,090.7 h), and the Q–Q plots in the replication package show the same pattern for every agent.

TABLE V
TIME-TO-CLOSE PER AGENT. CLOSED ISSUES: COUNT, MEDIAN IN HOURS WITH EXACT 95% CI, RANK. ALL ISSUES: % STILL OPEN, KAPLAN–MEIER (KM) MEDIAN IN DAYS (N.R. = 50% CLOSED NEVER REACHED), RANK

	Closed issues			All issues		Rank	
Agent	n	Median	95% CI	Open	KM	Closed	KM
PR-Agent	30	9.5	[1.1, 50.1]	45.5	152.9	1	9
Claude Code	694	11.1	[7.4, 15.7]	20.8	1.1	2	1
Sweep	463	22.5	[11.9, 36.8]	32.4	29.5	3	6
SWE-agent	243	26.6	[16.8, 57.9]	21.4	4.8	4	2
Gemini Code Assist	203	34.5	[17.8, 71.4]	20.1	4.8	5	3
Amazon Q	39	61.0	[22.9, 306.3]	63.6	n.r.	6	11
Aider	45	67.4	[13.9, 279.2]	33.8	79.7	7	7
Devin	308	75.0	[48.0, 160.7]	42.8	181.2	8	10
Kiro	46	105.0	[29.0, 339.4]	27.0	17.8	9	5
Jules	38	109.3	[45.8, 179.5]	22.4	7.0	10	4
CodeRabbit	1,160	180.3	[152.2, 252.2]	40.3	138.9	11	8

The comparison therefore uses the Kruskal–Wallis test with ε^2, Dunn's pairwise test with Holm correction and Cliff's δ. It is then repeated on all issues of the tested agents with the Kaplan–Meier estimator, which counts an issue that is still open as censored, so that it contributes the time it has been open without being treated as closed. Finally, the main test is repeated on the four sensitivity subsets S1–S4.

Results. *Finding 1: agents differ, but mostly at the extremes.* Time-to-close differs across agents (Kruskal–Wallis $H = 266.4$, df = 10, $p = 2.0 \times 10^{-51}$) with a medium effect ($\varepsilon^2 = 0.082$). Medians range from 9.5 hours for PR-Agent and 11.1 hours for Claude Code to 180.3 hours for CodeRabbit (Table V). The overall effect, however, comes from a few agents at the extremes. Only 13 of the 55 pairs of agents differ significantly after Holm correction, and by Cliff's δ, 28 pairs differ negligibly, 22 slightly, 5 moderately and 0 strongly. The largest difference is between Claude Code and CodeRabbit ($\delta = -0.44$), and Claude Code takes part in four of the five medium differences. The effect stays medium when issues closed as completed ($\varepsilon^2 = 0.097$) and issues closed as not planned ($\varepsilon^2 = 0.062$, 6 agents with at least ten such issues) are tested separately.

Robustness. The effect remains medium without pre-release issues (S1, $\varepsilon^2 = 0.083$), on the CodAGE precision subset (S3, $\varepsilon^2 = 0.092$) and with one value per agent and repository (S4, $\varepsilon^2 = 0.082$), so issues repeated in the same repository do not drive it. It shrinks to small when only human-authored issues are kept (S2, $\varepsilon^2 = 0.045$), which suggests that part of the difference between agents is associated with bot authorship. The ranking of the agents hardly changes in any subset (Spearman $\rho \geq 0.90$).

Finding 2: closed issues alone mislead. The picture changes once open issues are taken into account (Fig. 2). The difference between agents remains significant (log-rank $\chi^2 = 394.0$, $p = 1.8 \times 10^{-78}$), but the two rankings barely agree (Spearman $\rho = 0.22$). PR-Agent, the fastest agent on closed issues, drops to 9th, because 45.5% of its issues are still open. Amazon Q falls from 6th to last: 63.6% of its issues are open, so its Kaplan–Meier curve never reaches 50% closed. Jules moves the other way, from 10th to 4th. Claude Code is fast under both views, 2nd on closed issues and 1st with Kaplan–Meier (median 1.1 days).

The role of the agent again. The same bias affects the comparison of bot-written and other issues within an agent. Among *closed* issues, bot-written CodeRabbit issues take longer than other CodeRabbit issues (median 261.3 against 88.6 hours, $\delta = 0.19$), but bot-written Devin issues are closed *faster* than other Devin issues (54.0 against 361.6 hours, $\delta = -0.29$). Once open issues are counted, the bot-written issues of both agents are far slower: the Kaplan–Meier median is 202.8 against 11.1 days for CodeRabbit and 189.7 against 58.1 days for Devin (log-rank, Holm-adjusted $p = 5.2 \times 10^{-17}$ and 5.1×10^{-4}). For Devin, then, the closed issues alone would suggest the opposite of what happens to its issues as a whole.

RQ2 Summary. *Finding 1:* time-to-close differs across agents with a medium effect ($\varepsilon^2 = 0.082$), but 50 of the 55 pairs of agents differ negligibly or slightly; the differences lie at the extremes, with Claude Code fast and, on closed issues, CodeRabbit slow. *Takeaway:* adopters should not expect large speed differences between most agents. *Finding 2:* ranking agents by their closed issues disagrees with a survival analysis that counts open issues ($\rho = 0.22$), and for Devin the closed issues alone even reverse the comparison between bot-written and other issues. *Takeaway:* researchers and tool vendors should compare agent speed with survival analysis, not with closed issues only.

VI. RQ3: WHICH TOPICS DO THE ISSUES RAISE?

Motivation. RQ1 and RQ2 show *that* the agents' issues differ; they do not show what the issues are about. Studies of agent pull requests suggest that developers hand agents particular kinds of work, such as refactoring, documentation and testing [13]. If the same holds for issues, topics can explain part of the differences in closure and speed, and they tell maintainers what work to expect from each agent.

Approach. *Corpus.* The topic models cover the 4,203 English issues of the eleven tested agents. Each document is the issue title and body after removing code blocks, links, HTML, user mentions, template text (lines repeated verbatim in at least ten issues) and every agent name and signature, so that a topic cannot simply restate the agent label. Tokens were lemmatised and frequent word pairs joined, and only terms were kept that occur in at least ten documents and at most half of them. Documents with fewer than ten tokens were dropped, which leaves 3,816 documents (median 65 tokens) and 3,088 terms.

Single LDA runs. LDA [25] is the most widely used topic model in software-engineering research [26], [27], so the analysis started there. It was fitted for $k = 5$ to 20 topics with three random seeds each, and Cv coherence [47] was measured. Coherence hardly changes with k (0.47 to 0.51), so it cannot single out one model. A bigger problem is that the topics are not reproducible: when a model is refitted with another seed, at most 12% of the topics of the best model ($k = 8$, Cv = 0.524) and at most 23% for any k reappear, where a topic counts as reappearing if at least seven of its ten top words recur in the matched topic [29]. A topic list from

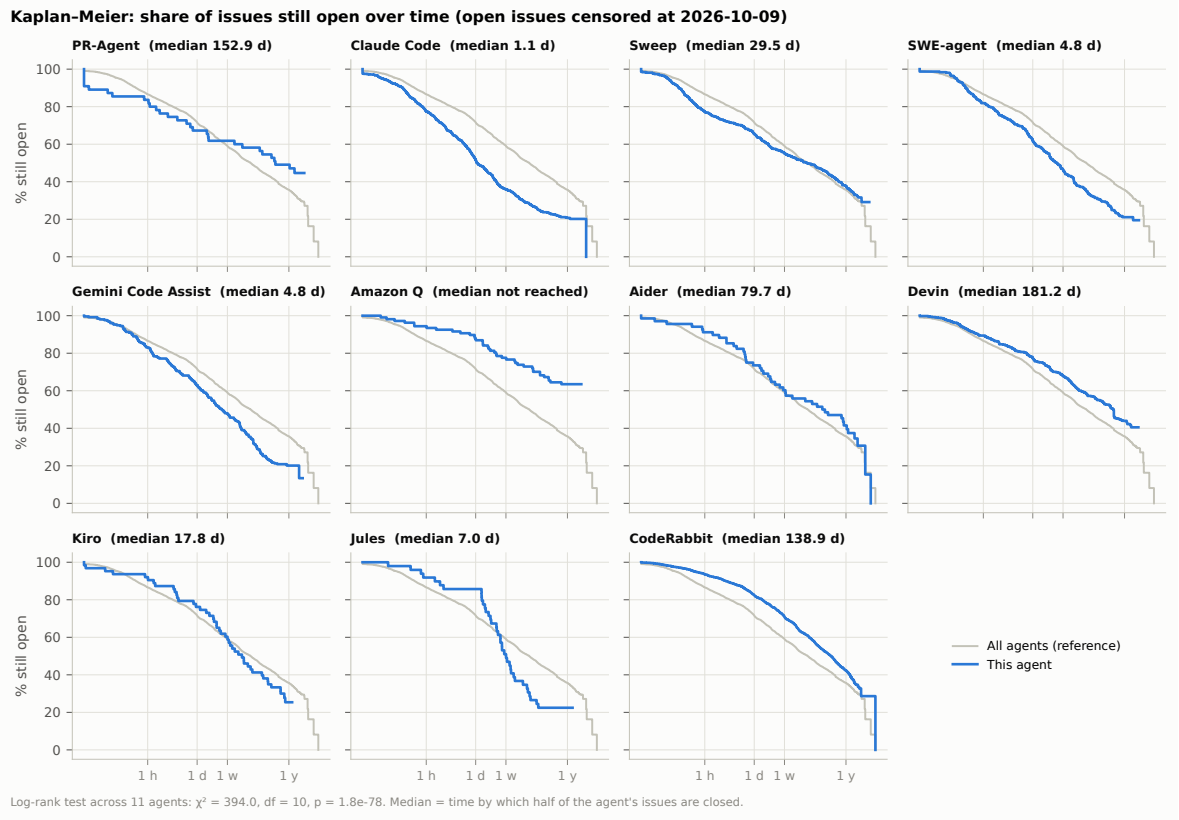


Fig. 2. Kaplan–Meier curves: share of each agent's issues still open over time, with open issues censored. The grey line shows all agents together.

a single run would therefore partly reflect the random seed, a risk that Agrawal et al. warned about for software-engineering data [28].

Ensemble LDA at two levels. The study therefore uses Ensemble LDA [48], which fits many models (here 16) and keeps only the topics that recur across them, with two independent ensembles fitted with different seeds. Under the rule fixed in advance, which chooses the clustering threshold that maximises the number of topics reproduced by both ensembles, 2 topics remain; they form the *headline* level. Two topics are coarse, so the paper also reports a *fine* view: the threshold at which both ensembles find the most topics (5). The researcher defined this second rule after seeing the headline result, so every fine topic is graded by how many of its ten top words recur in the matching topic of the other ensemble: *reproduced* (seven or more), *partly reproduced* (five or six) or *not reproduced* (fewer than five).

Labels and their validation. Each topic was labelled after reading its top words and its most representative issues, and the researcher then checked the labels of the fine topics on five random issues per topic, judging for each whether the label describes it. Before the check, the rule was set that a label fails if two or more of its five issues do not fit. A fine topic is interpreted only if it is reproduced *and* its label passes this check.

Triangulation with BERTopic. Recent studies of agent pull requests and of issue trackers use BERTopic [30]–[32], which clusters documents by meaning rather than by word co-occurrence. To test whether the conclusions depend on the method, the same documents were also clustered with BERTopic: sentence embeddings [49] of the whole issue (long issues are embedded in 150-word chunks that are then averaged), dimensionality reduction with UMAP [50], and k-means with as many clusters as the fine view has topics, repeated with three seeds.

Results. *Topics.* At the headline level, the issues split into code maintenance and quality work (H1) and AI and LLM tooling and research (H2), both reproduced across ensembles (Table VI). In the fine view, two topics meet both conditions for interpretation: testing, review and quality follow-ups (T1; label fits 4 of 5 sampled issues) and code-level fixes and refactoring (T4; 5 of 5). The other three fine topics are not interpreted. T2, whose words suggest implementation plans and integration phases, does not recur in the second ensemble. T3 and T5 recur only partly, and their labels failed the check (2 and 1 of 5 issues fit): the sampled issues ranged from a thermostat bug to a payment-channel problem, so these two word lists do not describe a coherent kind of work.

Finding 1: agents raise different kinds of work. The topics of an agent's issues depend on the agent (Fig. 3). The association is small at the headline level (V = 0.23) and at the border between medium and large in the fine view (V = 0.26, where

TABLE VI
RQ3 TOPICS AT THE HEADLINE LEVEL (H) AND IN THE FINE VIEW (T). SHARED = TOP-10 WORDS THAT RECUR IN THE MATCHING TOPIC OF THE SECOND, INDEPENDENT ENSEMBLE; PREV. = SHARE OF ALL TOPIC WEIGHT (%)

	Label	Shared	Prev.	Top words
H1	Code maintenance and quality work	8/10	75.1	`test, fix, add, validation, verify, error`
H2	AI/LLM tooling and research	9/10	24.9	`agent, model, llm, base, task, analysis`
T1	Testing, review and quality follow-ups	8/10	20.4	`test, fix, add, validation, error, review`
T2	*not interpreted*	1/10	14.4	`integration, phase, implement, test, performance, data`
T3	*not interpreted*	6/10	13.7	`agent, model, llm, tool, task, base`
T4	Code-level fixes and refactoring	8/10	28.5	`function, type, method, add, code, handle`
T5	*not interpreted*	6/10	23.0	`workflow, file, commit, update, create, add`

the threshold for a large effect is 0.25). It holds for human-authored issues only (S2, V = 0.22), on the precision subset (S3, V = 0.25) and when the documents of the non-reproduced topic are left out (V = 0.28). It is much weaker, though still significant, when the ensemble is refitted on a corpus that also contains very short issues of five to nine tokens: the topics then change, and V drops to 0.08 in the fine view and 0.12 at the headline level.

Which agents drive it. Adjusted residuals beyond $|z| = 3.32$ (Bonferroni) count as significant, and they are read only for the interpreted topics. CodeRabbit shows the clearest pattern. Its issues are strongly over-represented in code-level fixes (z = 19.5, and z = 17.8 on the precision subset), which make up 41.7% of the topic weight of its issues (95% bootstrap CI [40.2, 43.4]), and it wrote 67.6% of the issues dominated by this topic although it accounts for 43.7% of the documents. The pattern comes from the issues CodeRabbit writes itself: 92.2% of them refer to a pull request or a review, and code-level fixes make up 45.3% of their topic weight, against 20.1% for CodeRabbit's other issues. These are the issues that RQ1 showed are often left open.

Finding 2: the topic structure depends on the method. BERTopic is far more stable than LDA: all of its topics recur across seeds (adjusted Rand index between seeds 0.78 to 0.93). It confirms the main conclusion: topics depend on the agent, with a medium effect (V = 0.21, and 0.21 to 0.21 across seeds). On BERTopic's own topics, CodeRabbit is again over-represented in code-level work (z = 12.2). The two methods divide the corpus differently, however. Matched topics share few top words (mean Jaccard 0.34), and agreement at the level of single issues is low (adjusted Rand index 0.12; 37% of issues fall in the matched topic). The most plausible reading is that the differences between agents are strong enough to appear however the issues are grouped, while the issues themselves do not fall into a few sharply separated themes, so where one topic ends and the next begins depends largely on the model. The same reading fits the earlier observations that single LDA runs changed with the seed and that two of the fine topics failed the label check.

Topics and time-to-close. Time-to-close differs only slightly between the fine topics (ε^2 = 0.018, small; log-rank $p = 2.3 \times 10^{-11}$). Code-level fixes have the longest Kaplan–Meier median (83.1 days) and build and workflow issues the shortest (10.3 days). Because two-thirds of the code-level issues come from CodeRabbit, this difference cannot be separated from the agent.

RQ3 Summary. *Finding 1:* the topics of an agent's issues depend on the agent, under two topic models and on the S2 and S3 subsets; most clearly, CodeRabbit's own issues are follow-ups from pull-request reviews with code-level fixes as their largest topic. *Takeaway:* maintainers should expect review follow-up work from review agents, and researchers should treat "agent issues" as several kinds of work. *Finding 2:* only two of five fine topics are both reproducible and correctly labelled, and LDA and BERTopic divide the same issues differently while agreeing on the agent-level result. *Takeaway:* topic-modelling studies should check topic stability, validate topic labels by hand, and confirm their conclusions with a second method before interpreting individual topics.

VII. DISCUSSION

The three research questions point in the same direction. The agents differ, but much of the difference lies in the role the agent plays in an issue, and in whether the analysis looks only at issues that were closed. The findings matter to three audiences in different ways.

For maintainers. Issues written by an agent's own bot are, for both agents where a comparison is possible, closed far less often than the agent's other issues (RQ1). For the review agent CodeRabbit, these are follow-ups from its pull-request reviews, often code-level suggestions (RQ3). Projects that enable a review agent should therefore plan for this stream of issues: route them to a person who triages them, group them, or close them automatically after a set time. Otherwise they accumulate as backlog, the kind of bot activity that developers already describe as noise [7]. Maintainers should also not read the labels on issues of trigger-based agents such as Sweep as a classification of the work: there, most labels only invoke the agent (RQ1).

For agent developers. An agent that opens its own issues creates work for maintainers, and a large share of these issues stays open. Developers of such agents could make their issues easier to act on, for example by linking each issue to the review comment and the lines of code it comes from, by stating a priority, or by closing an issue once the underlying code has changed. Agents could also mark clearly how they are involved in an issue. That would reduce the ambiguity that currently forces datasets to attribute issues with heuristics [8], [15].

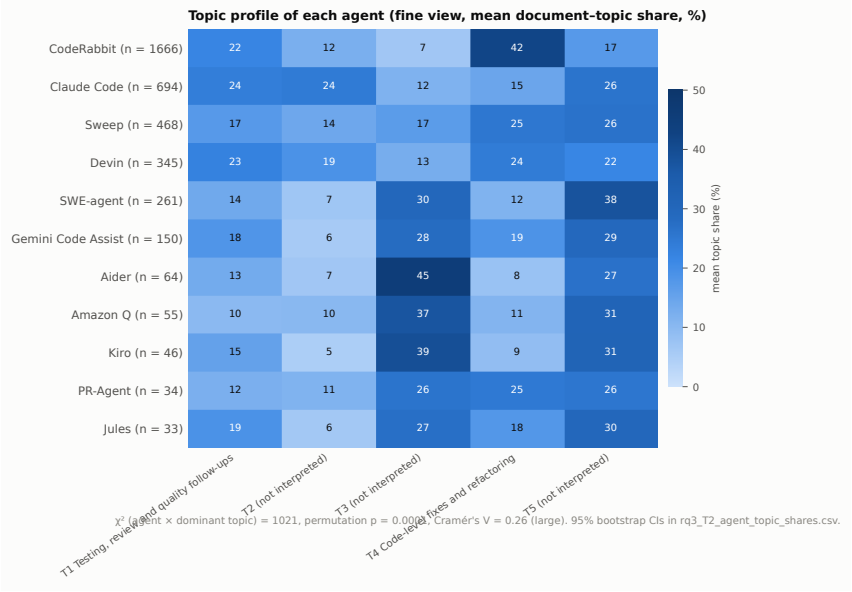


Fig. 3. Topic profile of each agent in the fine view: mean share of each topic in the agent's issues (%).

For researchers. "AI coding agent" is not one kind of actor. In the hand-checked sample, the agent had written fewer than a third of the issues correctly attributed to it, and in half it was only mentioned. Four consequences follow for studies that compare agents. First, they should record and report the role of the agent in each artefact, because pooling issues an agent wrote with issues that merely mention it can produce differences that belong to the role, not the agent (RQ1). They should also validate the attribution by hand on a sample; in this study, most wrong attributions came from text signals. Second, they should compare speed with survival analysis: in this study, the ranking of agents by closed issues barely agreed with the censoring-aware ranking, and for one agent the closed issues even reversed a comparison (RQ2). Third, they should remove agent-workflow labels before treating labels as data, adding to known problems with label quality [24]. Fourth, they should test whether their topics are reproducible, check the topic labels on sample issues, and confirm the results with a second method (RQ3): on this corpus, only two of five fine topics passed both the stability test and the label check. Single LDA runs were not reproducible on this corpus, which echoes earlier warnings [28], and two reasonable topic models divided the same issues differently while agreeing on the agent-level conclusions.

VIII. Threats to Validity

The four classic types of threats [51] are discussed below, each with the steps taken to reduce it.

Construct validity. The main construct, the agent behind an issue, comes from a heuristic attribution that its authors describe as a candidate with false positives by design [8]. It was kept as given, as the study brief requires, but the study reports for every agent how its issues are linked to it, repeats the key tests on the high-precision subset (S3) and after removing 10 issues created before the agent's public release (S1), and the researcher validated a stratified random sample of 99 issues by hand (nine per tested agent, rated blind). In this sample, 85.9% of the attributions are correct (95% CI [77.7, 91.4]), ranging from 5 of 9 for Aider to all nine for Claude Code, Devin and Kiro. Most errors (11 of 14) come from text signals, for example the word "Raider" matched as Aider, or the ordinary word "sweep" matched as the agent Sweep, which is the weakness the dataset's documentation warns about. The own-bot flag, on which the closure gap in RQ1 rests, was right for all 10 issues it flagged, but it caught only 10 of the 25 sampled issues that the agent itself had written (40.0%, 95% CI [23.4, 59.3]). It misses, for example, issues that people posted with a "Generated with Claude Code" footer. RQ1 therefore compares issues opened by the agent's own bot account with all of the agent's other issues, a group that also contains some agent-written and some misattributed issues. The rule was kept unchanged rather than tuned after seeing the validation. Some attributed issues are automated content that merely mentions an agent, for example 6 daily paper digests posted by `github-actions[bot]` that mention SWE-agent or Aider; the S3 subset excludes all but 1 of them. Time-to-close measures whether an issue was closed, not whether the problem was solved, and an issue closed as *not planned* is not a fix; each closure reason is therefore also tested separately. The label categories come from published rules defined for this study, the language flag is a heuristic that was checked by hand on 40 issues (19 of 20 flagged non-English and 20 of 20 flagged English issues were classified correctly), and the RQ3 topic labels are an interpretation of word lists and sample issues. They were checked by hand on five issues per topic, and only the two fine topics whose labels passed are interpreted (Section VI). All judgements were made

by the author alone; the blind samples and the answers are in the replication package, so that others can re-rate them.

Internal validity. This is an observational study, so all results are associations, not effects of the agents. Bot authorship is confounded with the agent: CodeRabbit and Devin issues are mostly bot-written and the others are not. Comparisons within the same agent (RQ1, RQ2) and the human-authored subset (S2) address this; in S2, the agent effect on time-to-close shrinks from medium to small. Issues of different agents also differ in age; age was checked and does not explain the closure gap, and the survival analysis accounts for the different time each issue has been open. Many issues from one repository could inflate the evidence, but no repository holds more than 12.7% of any tested agent's issues, and one value per agent and repository (S4) gives the same effect size.

External validity. The data are one extract of public GitHub issues for 13 agents, without any filter for repository popularity or activity, so personal and toy projects are included. The agents change quickly, and the results describe them as they were used in this period. Five of the tested agents have fewer than 110 issues, which limits what can be said about them individually, and RQ3 covers English issues only.

Conclusion validity. Time-to-close is heavily skewed, so all tests are rank-based, with an effect size reported for every test and a confidence interval with every rate and median. Families of tests are Holm-corrected, and sparse tables are tested by permutation. Statistical claims require at least 30 closed issues per agent; Selvanayagam and Ghaleb used a stricter threshold of 50 [16], which would remove five of the agents studied here. Single LDA runs were not reproducible on this corpus, so the topics come from Ensemble LDA, graded by their reproducibility, and the agent–topic association is confirmed with BERTopic. The fine-view rule and the "partly reproduced" grade were defined after the headline result; this is stated explicitly, and the headline level uses only the rule fixed in advance. The agent–topic association becomes weaker when very short issues are added. Finally, in gensim 4.3 the setting η ="auto" does not change the fitted topics, so fixed values of η were tuned instead. To guard against errors, every number in this report is generated from the result files, and independent scripts re-compute the key statistics.

IX. Conclusion and Future Work

This study compared 4,952 GitHub issues attributed to 13 AI coding agents on their volume, state and labelling, their time-to-close and their topics. The agents differ, but the clearest difference is in the role the agent plays: some agents write their own issues, others are triggered by humans or merely mentioned. That role is associated with what happens to the issue. Issues written by an agent's own bot are closed less often than the same agent's other issues; for the review agent CodeRabbit, these are follow-ups from its reviews. Comparing agents by their closed issues alone gives a misleading ranking, and survival analysis corrects it. Topics differ by agent under two topic models, even though the two models divide the corpus differently.

Future work. Three directions follow. *Following issues to their outcome:* linking agent-written issues to the pull requests and commits that address them would show whether open issues are ignored or resolved without being closed. *A human baseline:* comparing agent-attributed issues with human issues in the same repositories would show which differences are specific to agents. *Better attribution:* a larger hand-labelled sample, rated by more than one person, would allow the attribution to be corrected rather than only checked.

Ethics and Disclosures

Ethics. The study uses only public GitHub data provided for the course. The researcher followed the Menlo Report principles [52] and the guidance of Gold and Krinke on mining software repositories [53]: no attempt was made to identify individuals, only aggregates are reported, and no user accounts are named in this report. Agent names are product names. The replication package does not redistribute the dataset.

Use of generative AI. The researcher used Claude (Anthropic) as an assistant for the literature search, for writing and checking the code scripts, and for drafting and editing the text. All study decisions and the manual validation were the researcher's own; every reported number is generated from the result files and independently re-checked, and the researcher reviewed and is responsible for the entire content.

Data and Replication Package

The replication package is publicly available at https://github.com/Invincible1811/same-label-different-work. It contains all scripts (`src/`), the verification checks (`checks/`), every result table and the data behind every figure as CSV files, a manifest with SHA-256 hashes, exact library versions, the manual-validation samples and answers, the decision log, and a step-by-step guide. The course dataset is not redistributed; placing it in `data/raw/` and running `run_all.sh` reproduces all results. CodAGE is licensed under CC BY 4.0 and builds on GH Archive.

References

[1] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," arXiv:2405.15793, 2024.

[2] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2310.06770.

[3] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, "The impact of AI on developer productivity: Evidence from GitHub Copilot," arXiv:2302.06590, 2023.

[4] H. Li, H. Zhang, and A. E. Hassan, "AIDev: Studying AI coding agents on GitHub," in *Proc. 23rd International Conference on Mining Software Repositories (MSR)*, 2026, arXiv:2602.09185.

[5] T. F. Bissyandé, D. Lo, L. Jiang, L. Réveillère, J. Klein, and Y. Le Traon, "Got issues? who cares about it? a large scale investigation of issue trackers from GitHub," in *Proc. IEEE 24th International Symposium on Software Reliability Engineering (ISSRE)*, 2013, pp. 188–197.

[6] M. Wessel, A. Serebrenik, I. Wiese, I. Steinmacher, and M. A. Gerosa, "Effects of adopting code review bots on pull requests to OSS projects," in *Proc. IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2020.

[7] M. Wessel, I. Wiese, M. A. Gerosa, and I. Steinmacher, "Don't disturb me: Challenges of interacting with software bots on open source software projects," *Proc. ACM Hum.-Comput. Interact.*, vol. 5, no. CSCW2, pp. 301:1–301:21, 2021.

[8] T. A. Ghaleb, "CodAGE: A dataset of coding agent-generated GitHub events," https://huggingface.co/datasets/taher-ghaleb/CodAGE, 2026, cC BY 4.0.

[9] V. R. Basili, G. Caldiera, and H. D. Rombach, "The goal question metric approach," in *Encyclopedia of Software Engineering*. Wiley, 1994.

[10] R. Kikas, M. Dumas, and D. Pfahl, "Using dynamic and contextual features to predict issue lifetime in GitHub projects," in *Proc. 13th International Conference on Mining Software Repositories (MSR)*, 2016.

[11] H. Li, H. Zhang, and A. E. Hassan, "The rise of AI teammates in software engineering (SE) 3.0: How autonomous coding agents are reshaping software engineering," arXiv:2507.15003, 2025.

[12] G. Pinna, J. Gong, D. Williams, and F. Sarro, "Comparing AI coding agents: A task-stratified analysis of pull request acceptance," in *Proc. 23rd International Conference on Mining Software Repositories (MSR), Mining Challenge*, 2026, arXiv:2602.08915.

[13] M. Watanabe, H. Li, Y. Kashiwa, B. Reid, H. Iida, and A. E. Hassan, "On the use of agentic coding: An empirical study of pull requests on GitHub," *ACM Transactions on Software Engineering and Methodology*, 2025, accepted; arXiv:2509.14745.

[14] D. Pham and T. A. Ghaleb, "Code change characteristics and description alignment: A comparative study of agentic versus human pull requests," in *Proc. 23rd International Conference on Mining Software Repositories (MSR)*, 2026, arXiv:2601.17627.

[15] T. A. Ghaleb, "Fingerprinting AI coding agents on GitHub," in *Proc. 23rd International Conference on Mining Software Repositories (MSR)*, 2026, arXiv:2601.17406.

[16] N. Selvanayagam and T. A. Ghaleb, "AI-to-AI code reviews of GitHub pull requests," in *Proc. 20th International Symposium on Empirical Software Engineering and Measurement (ESEM)*, 2026, arXiv:2608.21311.

[17] M. Sayagh, "What makes a GitHub issue ready for Copilot?" arXiv:2512.21426, 2025.

[18] M. Wessel, B. M. de Souza, I. Steinmacher, I. S. Wiese, I. Polato, A. P. Chaves, and M. A. Gerosa, "The power of bots: Characterizing and understanding bots in OSS projects," *Proc. ACM Hum.-Comput. Interact.*, vol. 2, no. CSCW, pp. 182:1–182:19, 2018.

[19] M. Golzadeh, A. Decan, D. Legay, and T. Mens, "A ground-truth dataset and classification model for detecting bots in GitHub issue and PR comments," arXiv:2010.03303, 2021.

[20] E. Kalliamvakou, G. Gousios, K. Blincoe, L. Singer, D. M. German, and D. Damian, "The promises and perils of mining GitHub," in *Proc. 11th Working Conference on Mining Software Repositories (MSR)*, 2014, pp. 92–101.

[21] L. Erlenhov, F. Gomes de Oliveira Neto, and P. Leitner, "An empirical study of bots in software development: Characteristics and challenges from a practitioner's perspective," in *Proc. 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2020, pp. 445–455.

[22] J. Cabot, J. L. Cánovas Izquierdo, V. Cosentino, and B. Rolandi, "Exploring the use of labels to categorize issues in open-source software projects," in *Proc. IEEE 22nd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, 2015, pp. 550–554.

[23] R. Kallis, A. Di Sorbo, G. Canfora, and S. Panichella, "Ticket tagger: Machine learning driven issue classification," in *Proc. IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2019, pp. 406–409.

[24] K. Herzig, S. Just, and A. Zeller, "It's not a bug, it's a feature: How misclassification impacts bug prediction," in *Proc. 35th International Conference on Software Engineering (ICSE)*, 2013, pp. 392–401.

[25] D. M. Blei, A. Y. Ng, and M. I. Jordan, "Latent Dirichlet allocation," *Journal of Machine Learning Research*, vol. 3, pp. 993–1022, 2003.

[26] T.-H. Chen, S. W. Thomas, and A. E. Hassan, "A survey on the use of topic models when mining software repositories," *Empirical Software Engineering*, vol. 21, no. 5, pp. 1843–1919, 2016.

[27] C. C. Silva, M. Galster, and F. Gilson, "Topic modeling in software engineering research," *Empirical Software Engineering*, vol. 26, p. 120, 2021.

[28] A. Agrawal, W. Fu, and T. Menzies, "What is wrong with topic modeling? (and how to fix it using search-based software engineering)," *Information and Software Technology*, vol. 98, pp. 74–88, 2018.

[29] D. Greene, D. O'Callaghan, and P. Cunningham, "How many topics? stability analysis for topic models," in *Machine Learning and Knowledge Discovery in Databases (ECML PKDD)*, 2014, pp. 498–513.

[30] M. N. I. Opu, S. Islam, M. Asaduzzaman, and S. Chowdhury, "How do agentic AI systems address performance optimizations? A BERTopic-based analysis of pull requests," arXiv:2512.24630, 2025.

[31] J. K. Das, S. Mondal, and C. K. Roy, "Why do developers engage with ChatGPT in issue-tracker? Investigating usage and reliance on ChatGPT-generated code," arXiv:2412.06757, 2024.

[32] M. Grootendorst, "BERTopic: Neural topic modeling with a class-based TF-IDF procedure," arXiv:2203.05794, 2022.

[33] H. Cramér, *Mathematical Methods of Statistics*. Princeton University Press, 1946.

[34] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*, 2nd ed. Lawrence Erlbaum, 1988.

[35] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[36] W. H. Kruskal and W. A. Wallis, "Use of ranks in one-criterion variance analysis," *Journal of the American Statistical Association*, vol. 47, no. 260, pp. 583–621, 1952.

[37] M. Tomczak and E. Tomczak, "The need to report effect size estimates revisited. An overview of some recommended measures of effect size," *Trends in Sport Sciences*, vol. 21, no. 1, pp. 19–25, 2014.

[38] O. J. Dunn, "Multiple comparisons using rank sums," *Technometrics*, vol. 6, no. 3, pp. 241–252, 1964.

[39] N. Cliff, "Dominance statistics: Ordinal analyses to answer ordinal questions," *Psychological Bulletin*, vol. 114, no. 3, pp. 494–509, 1993.

[40] J. Romano, J. D. Kromrey, J. Coraggio, and J. Skowronek, "Appropriate statistics for ordinal level data: Should we really be using t-test and Cohen's d for evaluating group differences on the NSSE and other surveys?" in *Annual Meeting of the Florida Association of Institutional Research*, 2006.

[41] W. J. Conover, *Practical Nonparametric Statistics*, 3rd ed. Wiley, 1999.

[42] E. L. Kaplan and P. Meier, "Nonparametric estimation from incomplete observations," *Journal of the American Statistical Association*, vol. 53, no. 282, pp. 457–481, 1958.

[43] N. Mantel, "Evaluation of survival data and two new rank order statistics arising in its consideration," *Cancer Chemotherapy Reports*, vol. 50, no. 3, pp. 163–170, 1966.

[44] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[45] R. G. Newcombe, "Interval estimation for the difference between independent proportions: Comparison of eleven methods," *Statistics in Medicine*, vol. 17, no. 8, pp. 873–890, 1998.

[46] S. S. Shapiro and M. B. Wilk, "An analysis of variance test for normality (complete samples)," *Biometrika*, vol. 52, no. 3–4, pp. 591–611, 1965.

[47] M. Röder, A. Both, and A. Hinneburg, "Exploring the space of topic coherence measures," in *Proc. 8th ACM International Conference on Web Search and Data Mining (WSDM)*, 2015, pp. 399–408.

[48] T. Brigl, "Extracting reliable topics using ensemble latent Dirichlet allocation," Bachelor's thesis; method implemented as gensim's `EnsembleLda` (gensim 4.x documentation), 2019.

[49] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.

[50] L. McInnes, J. Healy, and J. Melville, "UMAP: Uniform manifold approximation and projection for dimension reduction," arXiv:1802.03426, 2018.

[51] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Springer, 2012.

[52] D. Dittrich and E. Kenneally, "The Menlo report: Ethical principles guiding information and communication technology research," U.S. Department of Homeland Security, Tech. Rep., 2012.

[53] N. E. Gold and J. Krinke, "Ethics in the mining of software repositories," *Empirical Software Engineering*, vol. 27, p. 17, 2022.